🧩 STEP 1 — Create and use a virtual environment

Install python 3.11 (because media pipe model that we will use doesn't work on python 3.13 and more)

Python version	Maintenance status		First released	End of support	Release schedule
3.15	pre-release	Download	2026-10-01 (planned)	2031-10	PEP 790
3.14	bugfix	Download	2025-10-07	2030-10	PEP 745
3.13	bugfix	Download	2024-10-07	2029-10	PEP 719
3.12	security	Download	2023-10-02	2028-10	PEP 693
3.11	security	Download	2022-10-24	2027-10	PEP 664
3.10	security	Download	2021-10-04	2026-10	PEP 619
3.9	end-of-life, last release was 3.9.25	Download	2020-10-05	2025-10-31	PEP 596

https://www.python.org/downloads/release/python-3110/

python.org/downloads/release/python-3110/

Terminer la mise à jour

Files

macOS

Download macOS 64-bit universal2 installer

Windows

Download Python install manager

Source release

Download XZ compressed source tarball

Version	Operating system	Description	File size	Sigstore		GPG	MD5 checksum
Gzipped source tarball	Source release		25.1 MB	.sigstore		SIG	c5f77f1ea256dc5bdb0897eeb4d35bb0
XZ compressed source tarball	Source release		18.9 MB	.sigstore		SIG	fe92acfa0db9b9f5044958edb451d463
macOS 64-bit universal2 installer	macOS	for macOS 10.9 and later	40.6 MB	CRT	SIG	SIG	98fa94815780c9330fc2154559365834
Windows installer (64-bit)	Windows	Recommended	24.0 MB	CRT	SIG	SIG	4fe11b2b0bb0c744cf74aff537f7cd7f
Windows installer (32-bit)	Windows		22.9 MB	CRT	SIG	SIG	e369a267acaad62487223bd835279bb9
Windows installer (ARM64)	Windows	Experimental	23.2 MB	CRT	SIG	SIG	18e5bd9a4854109adf3b77c7c9dc1ded
Windows embeddable package (64-bit)	Windows		10.1 MB	CRT	SIG	SIG	7df0f4244e5a66760b7caaed58e86c93
Windows embeddable package (32-bit)	Windows		9.1 MB	CRT	SIG	SIG	0888959642cc8af087d88da3866490a5
Windows embeddable package (ARM64)	Windows		9.3 MB	CRT	SIG	SIG	e3dbbd5d63c6cb203adc6c0c8ca5f5f7

🐍 Check Python version

```python
python --version
# for all versions installed in your device
py -0
```

📁 Create venv

```python
python -m venv venv
# Or this if you have multiple versions
py -3.11 -m venv venv
```

▶️ Activate venv

Windows:

```shell
venv\Scripts\activate
```

Linux / macOS:

```shell
source venv/bin/activate
```

👉 You should see:

(venv)

📦 Install basics

```shell
pip install opencv-python jupyter
# install this exact version
pip install mediapipe==0.10.9
```

🚀 Run Jupyter Lab

```shell
jupyter lab
```

🎥 STEP 2 — Open camera with OpenCV

```python
import cv2
cap = cv2.VideoCapture(0)

while True:
   ret, frame = cap.read()
   if not ret:
       break

   cv2.imshow("Camera", frame)

   if cv2.waitKey(1) == ord('q'):
       break

cap.release()
cv2.destroyAllWindows()
```

👉 Goal: just see camera working

👁️ STEP 3 — Add MediaPipe (face points)

Using MediaPipe

```python
import cv2
import mediapipe as mp

mp_face = mp.solutions.face_mesh
face_mesh = mp_face.FaceMesh()

cap = cv2.VideoCapture(0)

while True:
   ret, frame = cap.read()
   if not ret:
       break

   rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
   result = face_mesh.process(rgb)

   if result.multi_face_landmarks:
       for face in result.multi_face_landmarks:
           for point in face.landmark:
               h, w, _ = frame.shape
               x = int(point.x * w)
               y = int(point.y * h)

               cv2.circle(frame, (x, y), 1, (0, 255, 0), -1)

   cv2.imshow("Face Points", frame)
```

```
    if cv2.waitKey(1) == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

👉 Goal: show all face points

👁️ STEP 4 — (ONLY EYE POINTS)

```python
import cv2
import mediapipe as mp

mp_face = mp.solutions.face_mesh
face_mesh = mp_face.FaceMesh()

cap = cv2.VideoCapture(0)

OEIL_GAUCHE = [362, 385, 387, 263, 373, 380]

while True:
    ret, frame = cap.read()
    if not ret:
        break

    h, w, _ = frame.shape

    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    result = face_mesh.process(rgb)

    if result.multi_face_landmarks:
        for face in result.multi_face_landmarks:
```

```
            for i in OEIL_GAUCHE:
                point = face.landmark[i]
                x = int(point.x * w)
                y = int(point.y * h)
                cv2.circle(frame, (x, y), 3, (0, 0, 255), -1)

    cv2.imshow("Eye Points Only", frame)

    if cv2.waitKey(1) == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

📏 STEP 5 — (DISTANCE FUNCTION TEST)

```python
import math

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

def distance(p1, p2):
    return math.sqrt((p1.x - p2.x)**2 + (p1.y - p2.y)**2)

# Test
p1 = Point(1, 2)
p2 = Point(4, 6)
```

```
print("Distance:", distance(p1, p2))
```

🧮 STEP 6 — (EAR DISPLAY)

```python
import cv2
import mediapipe as mp
import math

mp_face = mp.solutions.face_mesh
face_mesh = mp_face.FaceMesh()

cap = cv2.VideoCapture(0)

OEIL_GAUCHE = [362, 385, 387, 263, 373, 380]

def distance(p1, p2):
   return math.sqrt((p1.x - p2.x)**2 + (p1.y - p2.y)**2)

def calculer_ear(points, indices):
   p1, p2, p3, p4, p5, p6 = [points[i] for i in indices]
   return (distance(p2, p6) + distance(p3, p5)) / (2.0 *
distance(p1, p4))

while True:
   ret, frame = cap.read()
   if not ret:
       break

   h, w, _ = frame.shape

   rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

```
    result = face_mesh.process(rgb)

    if result.multi_face_landmarks:
        for face in result.multi_face_landmarks:

            # draw eye
            for i in OEIL_GAUCHE:
                point = face.landmark[i]
                x = int(point.x * w)
                y = int(point.y * h)
                cv2.circle(frame, (x, y), 3, (0, 0, 255), -1)

            # EAR
            ear = calculer_ear(face.landmark, OEIL_GAUCHE)

            cv2.putText(frame, f"EAR: {ear:.2f}", (30, 30),
                        cv2.FONT_HERSHEY_SIMPLEX, 1, (0,255,0), 2)

    cv2.imshow("EAR", frame)

    if cv2.waitKey(1) == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

⏱️ STEP 7 — (FATIGUE TIMER)

```python
import cv2
import mediapipe as mp
import math
```

```python
import time

mp_face = mp.solutions.face_mesh
face_mesh = mp_face.FaceMesh()

cap = cv2.VideoCapture(0)

OEIL_GAUCHE = [362, 385, 387, 263, 373, 380]

def distance(p1, p2):
   return math.sqrt((p1.x - p2.x)**2 + (p1.y - p2.y)**2)

def calculer_ear(points, indices):
   p1, p2, p3, p4, p5, p6 = [points[i] for i in indices]
   return (distance(p2, p6) + distance(p3, p5)) / (2.0 *
distance(p1, p4))

start_time = None

while True:
   ret, frame = cap.read()
   if not ret:
       break

   h, w, _ = frame.shape

   rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
   result = face_mesh.process(rgb)

   if result.multi_face_landmarks:
       for face in result.multi_face_landmarks:

           ear = calculer_ear(face.landmark, OEIL_GAUCHE)

           cv2.putText(frame, f"EAR: {ear:.2f}", (30, 30),
                       cv2.FONT_HERSHEY_SIMPLEX, 1, (0,255,0), 2)
```

```
            if ear < 0.25:
                if start_time is None:
                    start_time = time.time()

                duration = time.time() - start_time

                cv2.putText(frame, f"Closed: {duration:.1f}s",
(30, 60),
                            cv2.FONT_HERSHEY_SIMPLEX, 1,
(0,0,255), 2)

                if duration >= 2:
                    cv2.putText(frame, "SLEEP!", (30, 100),
                                cv2.FONT_HERSHEY_SIMPLEX, 1,
(0,0,255), 3)
            else:
                start_time = None

    cv2.imshow("Fatigue Detection", frame)

    if cv2.waitKey(1) == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

🔊 STEP 8 — (WITH SOUND)

```python
import cv2
import mediapipe as mp
```

```python
import math
import time
import pygame

pygame.mixer.init()
pygame.mixer.music.load("alarm.mp3")

mp_face = mp.solutions.face_mesh
face_mesh = mp_face.FaceMesh()

cap = cv2.VideoCapture(0)

OEIL_GAUCHE = [362, 385, 387, 263, 373, 380]

def distance(p1, p2):
   return math.sqrt((p1.x - p2.x)**2 + (p1.y - p2.y)**2)

def calculer_ear(points, indices):
   p1, p2, p3, p4, p5, p6 = [points[i] for i in indices]
   return (distance(p2, p6) + distance(p3, p5)) / (2.0 *
distance(p1, p4))

start_time = None
alarm_on = False

while True:
   ret, frame = cap.read()
   if not ret:
       break

   rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
   result = face_mesh.process(rgb)

   if result.multi_face_landmarks:
       for face in result.multi_face_landmarks:
```

```
            ear = calculer_ear(face.landmark, OEIL_GAUCHE)

            if ear < 0.25:
                if start_time is None:
                    start_time = time.time()

                duration = time.time() - start_time

                if duration >= 2:
                    cv2.putText(frame, "SLEEP!", (30, 100),
                                cv2.FONT_HERSHEY_SIMPLEX, 1,
(0,0,255), 3)

                    if not alarm_on:
                        pygame.mixer.music.play(-1)
                        alarm_on = True
            else:
                start_time = None

                if alarm_on:
                    pygame.mixer.music.stop()
                    alarm_on = False

    cv2.imshow("Alarm System", frame)

    if cv2.waitKey(1) == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
pygame.mixer.quit()
```

🔊 STEP 9 *with Arduino and buzzer*

```
import cv2
import mediapipe as mp
import math
import time
import serial

# ---------- Arduino ----------
arduino = serial.Serial('COM3', 9600)
time.sleep(2)

def buzzer_on():
   arduino.write(b'1')

def buzzer_off():
   arduino.write(b'0')

# ---------- MediaPipe ----------
mp_face = mp.solutions.face_mesh
detecteur = mp_face.FaceMesh()

# ---------- Camera ----------
camera = cv2.VideoCapture(0)

# ---------- Parameters ----------
SEUIL_EAR = 0.25
SEUIL_TEMPS_YEUX = 2.0

OEIL_GAUCHE = [362, 385, 387, 263, 373, 380]
OEIL_DROIT  = [33, 160, 158, 133, 153, 144]

# ---------- Utils ----------
def distance(p1, p2):
   return math.sqrt((p1.x - p2.x)**2 + (p1.y - p2.y)**2)

def calculer_ear(points, indices):
   p1, p2, p3, p4, p5, p6 = [points[i] for i in indices]
   return (distance(p2, p6) + distance(p3, p5)) / (2.0 * distance(p1,
p4))
```

```
# ---------- Variables ----------
temps_yeux = None
alarme_active = False

# ---------- Main Loop ----------
while True:
   ret, image = camera.read()
   if not ret:
       break

   image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
   resultats = detecteur.process(image_rgb)

   if resultats.multi_face_landmarks:
       for visage in resultats.multi_face_landmarks:
           points = visage.landmark

           ear_g = calculer_ear(points, OEIL_GAUCHE)
           ear_d = calculer_ear(points, OEIL_DROIT)
           ear = (ear_g + ear_d) / 2.0

           if ear < SEUIL_EAR:
               if temps_yeux is None:
                   temps_yeux = time.time()

               duree = time.time() - temps_yeux

               if duree >= SEUIL_TEMPS_YEUX:
                   cv2.putText(image, "SLEEP DETECTED!", (50, 100),
                               cv2.FONT_HERSHEY_SIMPLEX, 1,
(0,0,255), 3)

                   if not alarme_active:
                       buzzer_on()
                       alarme_active = True
           else:
               temps_yeux = None

               if alarme_active:
                   buzzer_off()
                   alarme_active = False

   cv2.imshow("DriveSafe", image)

   if cv2.waitKey(1) == ord('q'):
```

```
        break

# ---------- Cleanup ----------
camera.release()
cv2.destroyAllWindows()
buzzer_off()
arduino.close()

print("Done")
```